ROBERT GORDON UNIVERSITY ABERDEEN

CM3604 – Deep Learning

Academic Year	**2025/2026**
Semester	**Semester 1**
Module Number	**CM3604**
Module Title	**Deep Learning**
Assessment Method	**Coursework Element 2 – Group**
Deadline (time and date)	**16th November 2025 23:59**
Submission	**Assessment Dropbox in the Module Study Area in CampusMoodle.**
Word Limit (see Assessment Word Limit Statement)	**N/A**
Module Co-ordinator	**Prasan Yapa**

What knowledge and/or skills will I develop by undertaking the assessment?

The primary goal with this task is to give you hands-on experience implementing a Transformer language model. Understanding how these neural models work and building one from scratch will help you understand not just language modelling, but also systems for many other applications such as machine translation. You may form a group of 3 to 4 members for this task and only one member should make the submission.

On successful completion of the assessment students will be able to achieve the following Learning Outcomes:

1. Build a neural model by selecting relevant functions and parameters using a state-of-the-art python framework.
2. Implement, test and transfer deep architectures for a given data science application.

Please also refer to the Module Descriptor, available from the module Moodle study area.

What is expected of me in this assessment?

Task(s) – content

Please use Python 3.5+ and PyTorch 1.0+ for this project. The dataset for this project is the Text collection by Matt Mahoney. This is a dataset taken from the first 100M characters of Wikipedia. Only 27-character types are present (lowercase characters and spaces); special characters are replaced by a single space and numbers are spelled out as individual digits (20 becomes two zero). A larger version of this benchmark (90M training characters, 5M dev, 5M test) was used in Mikolov et al. (2012).

The framework code you are given consists of several files. *letter_counting.py* contains the driver for Part 1, which imports *transformer.py*. *lm.py.* contains the driver for Part 2 and imports *transformer_lm.py*. *utils.py* contains all the required utility logic to the framework.

Task(s) – format

Part 1: Building a Transformer Encoder (50 Marks)

In this first part, you will implement a simplified Transformer (missing components like layer normalization and multi-head attention) from scratch for a simple task. **Given a string of characters, your task is to predict, for each position in the string, how many times the character at that position occurred before, maxing out at 2**. This is a 3-class classification task (with labels 0, 1, or > 2 which we'll just denote as 2). This task is easy with a rule-based system, but it is not so easy for a model to learn. However, Transformers are ideally set up to be able to "*look back*" with self-attention to count occurrences in the context. Below is an example string (which ends in a trailing space) and its corresponding labels:

I	like	movies	a	lot
00	01001	0002102	02	1102

We also present a modified version of this task that counts both occurrences of letters before *and after* in the sequence:

What is expected of me in this assessment?

I	like	movies	a	lot
22	12012	0102102	02	1102

Note that every letter of the same type always receives the same label no matter where it is in the sentence in this version. Adding the --task BEFOREAFTER flag will run this second version; default is the first version.

Data *lettercounting-train.txt* and *lettercounting-dev.txt* both contain character strings of length 20. You can assume that your model will always see 20 characters as input. You need to make a prediction at each position in the sequence.

Getting started Run: *python letter_counting.py* --task BEFOREAFTER

This loads the data for this part but will fail out because the Transformer hasn't been implemented yet. Implement Transformer and TransformerLayer for the BEFOREAFTER version of the task. You should identify the number of other letters of the same type in the sequence. This will require implementing both Transformer and TransformerLayer, as well as training in *train_classifier*. You should only use Linear, Softmax, and standard nonlinearities to implement Transformers from scratch. **You are not allowed to use nn.TransformerEncoder, nn.TransformerDecoder, or any other off-the-shelf self-attention layers.**

TransformerLayer: This layer should follow the format: (1) self-attention (single-headed is fine; you can use either backward-only or bidirectional attention); (2) residual connection; (3) linear layer, nonlinearity, and Linear layer; (4) final residual connection. With a shallow network like this, you don't need layer normalization, which is a bit more complicated to implement. You will want to form queries, keys, and values matrices with linear layers, then use the queries and keys to compute attention over the sentence, then combine with the values. You'll want to use *matmul* for this purpose, and you may need to transpose matrices as well.

What is expected of me in this assessment?

Transformer: Building the Transformer will involve: (1) adding positional encodings to the input (2) using one or more of the TransformerLayers; (3) using Linear and Softmax layers to make the prediction. Your network should return the log probabilities at the output layer (a 20x3 matrix) as well as the attentions you compute, which are then plotted for you for visualization purposes in *plots/*.

Training: A skeleton is provided in *train_classifier*. Without positional encodings, your model may struggle a bit, but you should be able to get at least 85% accuracy with a single-layer Transformer in a few epochs of training. The attention maps should also show some evidence of the model attending to the characters in context.

Q1 (40 Marks) Now extend your Transformer classifier with positional encodings and address the main task: identifying the number of letters of the same type **preceding** that letter. Run this with *python letter_counting.py*, no other arguments. Without positional encodings, the model simply sees a bag of characters and cannot distinguish letters occurring later or earlier in the sentence.

Overall: Your final implementation should get over **95% accuracy** on this task. Also note that **the autograder trains your model on an additional task as well**. You will fail this hidden test if your model uses anything hardcoded about these labels (or if you try to cheat and just return the correct answer that you computed by directly counting letters yourself), but any implementation that works for this problem will work for the hidden test.

Debugging Tips: As always, make sure you can overfit a very small training set as an initial test, inspecting the loss of the training set at each epoch. You will need your learning rate set carefully to let your model train. Even with a good learning rate, it will take longer to overfit data with this model than with others we've explored. Consider using small values for hyperparameters so things train quickly. In particular, with only 27 characters, you can get away with small embedding sizes for these, and small hidden sizes for the Transformer (100 or less) may work better than you think.

What is expected of me in this assessment?

Q2 (10 Marks) Look at the attention masks produced. What is the model doing? Does it match your expectations? Try using more Transformer layers (3-4). Do all of the attention masks fit the pattern you expect?

Part 2: Transformer for Language Modelling (50 Marks)

In this section, you will implement a Transformer language model. This should build heavily off of what you did for Part 1, although for this part you are allowed to use off-the-shelf Transformer components. For this part, we use the first 100,000 characters of *text8* as the training set. The development set is 500 characters taken from elsewhere in the collection. Your model will need to be able to consume a chunk of characters and make predictions of the next character at each position simultaneously. Structurally, this looks exactly like Part 1, although with 27 output classes instead of 3.

Getting started Run: *python lm.py*

This loads the data, instantiates a *UniformLanguageModel* which assigns each character an equal $\frac{1}{27}$ probability, and evaluates it on the development set. This model achieves a total log probability of -1644, an average log probability (per token) of -3.296, and a perplexity of 27. Note that exponentiating the average log probability gives you $\frac{1}{27}$ in this case, which is the inverse of perplexity. The NeuralLanguageModel class you are given has one method: *get_next_char_log_probs*. It takes a context and returns the log probability distribution over the next characters given that context as a NumPy vector of length equal to the vocabulary size.

Q3 (50 Marks) Implement a Transformer language model. This will require defining a PyTorch module to handle language model prediction, implementing training of that module in *train_lm*, and finally completing the definition of *NeuralLanguageModel* appropriately to use this module for prediction. Your network should take a chunk of indexed characters as input, embed them, put them through a Transformer, and make predictions from the final layer outputs. Your final model must pass the sanity and

What is expected of me in this assessment?

normalization checks, get a perplexity value less than or equal to 7, and train in less than 10 minutes. In your report, you should: (1) Describe your model and implementation. (2) Report accuracy.

Network structure: You can use a similar input layer as in Part 1 to encode the character indices. You can use the PositionalEncoding from Part 1. You can then use your Transformer architecture from Part 1 or you can use a real nn.TransformerEncoder, which is made up of TransformerEncoderLayers. Note that unlike the Transformer encoder you used in part 1, for Part 2 you must be careful to use a causal mask for the attention: tokens should not be able to attend to tokens occurring after them in the sentence, or else the model can easily "*cheat*". Try to use your Transformer from Part 1 for Part 2. Note: you will probably have to implement multi-head self-attention in order to get it to work well, so you may want to experiment with that as well.

Training on chunks: Unlike in Part 1, you are presented with data in a long, continuous stream of characters. Nevertheless, your network should process a chunk of characters at a time, simultaneously predicting the next character at each index in the chunk. You'll have to decide how you want to chunk the data for both training and inference. Given a chunk, you can either train just on that chunk or include a few extra tokens for context and not compute loss over those positions.

Tensor manipulation: *np.asarray* can convert lists into NumPy arrays easily. *torch.from_numpy* can convert NumPy arrays into PyTorch tensors. *torch.FloatTensor(list)* can convert from lists directly to PyTorch tensors. *.float()* and *.int()* can be used to cast tensors to different types. *unsqueeze* allows you to add trivial dimensions of size 1, and *squeeze* lets you remove these.

Evaluation: The model should be evaluated on perplexity and likelihood. Your model must be a **correct** implementation of a language model. Correct in this case means that it must represent a probability distribution. You should be sure to check that your model's output is indeed a legal probability distribution over the next word.

What is expected of me in this assessment?

Submission: You should submit the following files separately.

1. A PDF of your answers to the questions.
2. *transformer.py* and *transformer_lm.py*. **Do not modify or upload any other source files.**

Make sure that the following commands work before you submit:

python letter_counting.py

python lm.py --model NEURAL

These commands should run without errors.

How will I be graded?	
A grade will be provided for each criterion on the feedback grid which is specific to the assessment. The overall grade for the assessment will be calculated using the algorithm below.	
A	At least 50% of the feedback grid to be at Grade A, at least 75% of the feedback grid to be at Grade B or better, and normally 100% of the feedback grid to be at Grade C or better.
B	At least 50% of the feedback grid to be at Grade B or better, at least 75% of the feedback grid to be at Grade C or better, and normally 100% of the feedback grid to be at Grade D or better.
C	At least 50% of the feedback grid to be at Grade C or better, and at least 75% of the feedback grid to be at Grade D or better.
D	At least 50% of the feedback grid to be at Grade D or better, and at least 75% of the feedback grid to be at Grade E or better.
E	At least 50% of the feedback grid to be at Grade E or better.
F	Failing to achieve at least 50% of the feedback grid to be at Grade E or better.
NS	Non-submission.

ROBERT GORDON UNIVERSITY ABERDEEN

Feedback grid

GRADE	A	B	C	D	E	F
DEFINITION / CRITERIA (WEIGHTING)	**EXCELLENT** Outstanding Performance	**COMMENDABLE/VERY GOOD** Meritorious Performance	**GOOD** Highly Competent Performance	**SATISFACTORY** Competent Performance	**BORDERLINE FAIL**	**UNSATISFACTORY** Fail
Transformer Encoder **(40%)** **Grade:** ☐	Excellent use of an optimal network structure, proper usage of embeddings selecting optimal tensor dimensions and feeding the resulting tensor into the selected network, proper usage of layer normalization and activation functions from scratch, demonstrating multi-head attention and computing the loss, excellent showcase of hyper-parameter settings and excellent error/exception handling.	Very good use of an optimal network structure, proper usage of embeddings selecting optimal tensor dimensions and feeding the resulting tensor into the selected network, very good usage of layer normalization and activation functions from scratch, demonstrating multi-head attention and computing the loss, very good showcase of hyper-parameter settings and commendable error/exception handling.	Good use of an optimal network structure, proper usage of embeddings selecting optimal tensor dimensions and feeding the resulting tensor into the selected network, good usage of layer normalization and activation functions from scratch, demonstrating multi-head attention and computing the loss, good showcase of hyper-parameter settings and error/exception handling.	Satisfactory use of an optimal network structure, usage of embeddings selecting tensor dimensions and feeding the resulting tensor into the selected network, satisfactory usage of layer normalization and activation functions from scratch, demonstrating multi-head attention and computing the loss, showcase of hyper-parameter settings and error/exception handling.	Lacks understanding on the use of an optimal network structure, usage of embeddings selecting tensor dimensions and feeding the resulting tensor into the selected network, improper usage of layer normalization and activation functions from scratch, poorly demonstrating multi-head attention and computing the loss, hyper-parameter settings and error/exception handling.	No or very limited evidence in using an optimal network structure, usage of embeddings selecting tensor dimensions and feeding the resulting tensor into the selected network, poor usage of layer normalization and activation functions from scratch, no evidence in demonstrating multi-head attention and computing the loss, hyper-parameter settings and error/exception handling.
Exploration **(10%)** **Grade:** ☐	Excellent understanding of using different values and combinations for the number of Transformer layers and attention masks to fit with the expected patterns.	Very good understanding of using different values and combinations for the number of Transformer layers and attention masks to fit with the expected patterns.	Highly competitive understanding of using different values and a few combinations for the number of Transformer layers and attention masks to fit with the expected patterns.	Satisfactory usage of different values and a very few combinations for the number of Transformer layers and attention masks to fit with the expected patterns.	Lacks understanding on the usage of different values and combinations for the number of Transformer layers and attention masks to fit with the expected patterns.	No or very limited evidence in understanding the usage of different values and combinations for the number of Transformer layers and attention masks to fit with the expected patterns.

GRADE	A	B	C	D	E	F
DEFINITION / CRITERIA (WEIGHTING)	**EXCELLENT** Outstanding Performance	**COMMENDABLE/VERY GOOD** Meritorious Performance	**GOOD** Highly Competent Performance	**SATISFACTORY** Competent Performance	**BORDERLINE FAIL**	**UNSATISFACTORY** Fail
Transformer Language Model **(30%)** **Grade:** ☐	Outstanding performance and understanding to process a chunk of indexed characters as input, embed them using tensor manipulation, put them through a Transformer by incorporating a casual mask for the attention, and make predictions from the final layer outputs.	Very good understanding to process a chunk of indexed characters as input, embed them using tensor manipulation, put them through a Transformer by incorporating a casual mask for the attention, and make predictions from the final layer outputs.	Highly competitive understanding to process a chunk of indexed characters as input, embed them using tensor manipulation, put them through a Transformer by incorporating a casual mask for the attention, and make predictions from the final layer outputs.	Competitive performance to process a chunk of indexed characters as input, embed them using tensor manipulation, put them through a Transformer by incorporating a casual mask for the attention, and make predictions from the final layer outputs.	Lacks understanding to process a chunk of indexed characters as input, embed them using tensor manipulation, put them through a Transformer by incorporating a casual mask for the attention, and make predictions from the final layer outputs.	No or very limited evidence to process a chunk of indexed characters as input, embed them using tensor manipulation, put them through a Transformer by incorporating a casual mask for the attention, and make predictions from the final layer outputs.
Model Evaluation **(20%)** **Grade:** ☐	Outstanding performance and understanding to evaluate the language model on perplexity and likelihood, execution of commands without errors, and documentation.	Very good understanding to evaluate the language model on perplexity and likelihood, execution of commands without errors, and documentation.	Good understanding to evaluate the language model on perplexity and likelihood, execution of commands without errors, and documentation.	Satisfactory understanding to evaluate the language model on perplexity and likelihood, execution of commands without errors, and documentation.	Lacks understanding to evaluate the language model on perplexity and likelihood, execution of commands without errors, and documentation.	No or very limited evidence to evaluate the language model on perplexity and likelihood, execution of commands without errors, and documentation.

Coursework received late, without valid reason, will be regarded as a non-submission (NS) and one of your assessment opportunities will be lost.

What else is important to my assessment?

What is plagiarism?

"Plagiarism is the practice of presenting the thoughts, writings or other output of another or others as original, without acknowledgement of their source(s) at the point of their use in the student's work. All materials including text, data, diagrams or other illustrations used to support a piece of work, whether from a printed publication or from electronic media, should be appropriately identified and referenced and should not normally be copied directly unless as an acknowledged quotation. Text, opinions or ideas translated into the words of the individual student should in all cases acknowledge the original source" (RGU 2022).

What is collusion?

"Collusion is defined as two or more people working together with the intention of deceiving another. Within the academic environment this can occur when students work with others on an assignment, or part of an assignment, that is intended to be completed separately" (RGU 2022).

For further information please see Academic Integrity.

What is the Assessment Word Limit Statement?

It is important that you adhere to the Word Limit specified above. The Assessment Word Limit Statement lists what is included and excluded from the word count, along with the penalty for exceeding the upper limit.

What if I'm unable to submit?

- The University operates a Fit to Sit Policy which means that if you undertake an assessment then you are declaring yourself well enough to do so.
- If you require an extension, you should complete and submit a Coursework Extension Form. This form is available on the RGU Student and Applicant Forms page.
- Further support is available from your Course Leader.

What else is important to my assessment?

What additional support is available?

- RGU Study Skills provide advice and guidance on academic writing, study skills, maths and statistics and basic IT.
- RGU Library guidance on referencing and citing.
- The Inclusion Centre: Disability & Dyslexia.
- Your Module Coordinator, Course Leader and designated Personal Tutor can also provide support.

What are the University rules on assessment?

The University Regulation 'A4: Assessment and Recommendations of Assessment Boards' sets out important information about assessment and how it is conducted across the University.